

1. ¿Cuál es la función de la clase NodeTree?

Representar cada nodo o elemento dentro del árbol binario.

2. ¿Qué representa el atributo root?

Representa la raíz del árbol.

3. ¿Qué significa que un nodo tenga como hijo el valor None?

Significa que inicialmente o en esa posición el nodo no tiene hijos.

4. ¿Cuál es el caso base del método _insertar?

El caso base es if node is None:.

5. ¿Qué sucede cuando _insertar encuentra una posición vacía?

Se crea y se devuelve un nuevo nodo (return NodeTree(value)).

6. ¿Por qué se utiliza return node al final del método _insertar?

Para permitir conservar las conexiones entre los nodos del árbol.

7. ¿Qué sucede cuando se intenta insertar un valor repetido?

Muestra el mensaje "El valor ya existe en el árbol." y no se crea un nodo duplicado.

8. ¿Cómo decide el método buscar si debe continuar a la izquierda o a la derecha?

Utiliza el orden del árbol: si el valor buscado es menor que el nodo actual (value < node.value), continúa hacia la izquierda; si es mayor, continúa hacia la derecha.

9. ¿Cuál es el caso base de la búsqueda?

El caso base es cuando el nodo actual es None (if node is None: return False), lo que significa que el valor no se encuentra.

10. ¿Cuál es el caso base de los recorridos?

El caso base es if node is not None:.. Cuando el método llega a una posición vacía (None), deja de realizar llamadas recursivas.

11. ¿En qué momento se muestra la raíz en el recorrido preorden?

Se muestra al principio (sigue la secuencia: Raíz -> Izquierda -> Derecha).

12. ¿En qué momento se muestra la raíz en el recorrido inorden?

Se muestra entre el subárbol izquierdo y el subárbol derecho (sigue la secuencia: Izquierda -> Raíz -> Derecha).

13. ¿En qué momento se muestra la raíz en el recorrido posorden?

Se muestra al final (sigue la secuencia: Izquierda -> Derecha -> Raíz).

14. ¿Por qué el recorrido inorden muestra los valores ordenados?

Porque en un árbol binario de búsqueda los valores menores están a la izquierda y los mayores a la derecha; al recorrer en orden Izquierda -> Raíz -> Derecha, los elementos quedan mostrados de menor a mayor.

15. ¿Qué sucede si los valores se insertan de menor a mayor?

El árbol pierde una forma equilibrada y se parece a una lista enlazada.

16. ¿Cuál es la diferencia entre el while del menú y las llamadas utilizadas dentro de la clase BinaryTree?

El while True del programa principal no forma parte del funcionamiento interno del árbol, sino que se usa para mantener el menú activo y permitir múltiples operaciones; mientras que las llamadas dentro de BinaryTree son recursivas para desplazarse por los nodos.

17. ¿Qué parte del programa evita que se ingresen letras cuando se solicita un número?

El bloque try ... except ValueError: en el menú de consola, el cual captura el error y muestra el mensaje "Debes ingresar un número entero.".

18. ¿Por qué la forma del árbol depende del orden de inserción?

Porque el primer elemento insertado se convierte en la raíz y cada elemento posterior se coloca a la izquierda o derecha según si es menor o mayor que los nodos existentes.

19. ¿Por qué desde el menú no se debe acceder directamente a tree.root?

Para que el programa principal utilice el árbol sin acceder directamente a sus atributos internos, delegando a la clase BinaryTree la responsabilidad de iniciar los recorridos o búsquedas desde la raíz mediante sus métodos públicos.

20. ¿Qué función cumplen los métodos _preorden, _inorden y _posorden?

Son métodos auxiliares que reciben cada nodo para desplazarse por el árbol realizando el recorrido correspondiente mediante llamadas recursivas.